

On Data-Driven Self-Calibration for IoT-Based Gas Concentration Monitoring Systems

Yang You¹, *Student Member, IEEE*, Kai You, Hao Chen², *Student Member, IEEE*,
and Tobias J. Oechtering³, *Senior Member, IEEE*

Abstract—In this article, data-driven self-calibration algorithms for the Internet of Things-based gas concentration monitoring systems embedded with low-cost gas sensors are designed. The measurement errors are assumed to be caused by imperfect compensation for the variation of sensor component behavior. Specifically, the calibration procedure for the nondispersive infrared CO_2 sensors is developed, for which the temperature dependency is the most dominant drift source. For a single sensor, the hidden Markov model is used to characterize the statistical relationship between different quantities introduced by the physical model that builds on the Beer–Lambert law. For the calibration in the Internet of Things-based system, sensors first transmit their belief functions of the true gas concentration level to the cloud. Then, the cloud fusion center computes a fused belief function according to certain rules. This belief function is then used as reference for calibrating the sensors. To deal with the case where belief functions highly conflict with each other, a Wasserstein distance-based weighted average belief function fusion approach is first proposed as a networked calibration algorithm. To achieve more long-term stable calibration results, the networked calibration problem is further formulated as a partially observed Markov decision process (MDP) problem, and the calibration strategies are derived in a sequential manner. Correspondingly, the deep Q -network approach is applied as a computationally efficient method to solve the proposed MDP problem. The performance of practical designs of the proposed self-calibration algorithms is finally illustrated in numerical experiments utilizing real data.

Index Terms—Belief function fusion, data-driven modeling, deep reinforcement learning (DRL), Internet of Things (IoT), nondispersive infrared (NDIR) gas sensor calibration, partially observed Markov decision process (POMDP).

I. INTRODUCTION

IN RECENT years, there has been a growing interest in deploying Internet of Things (IoT) systems for gas concentration monitoring [1]–[4]. For example, IoT-based gas concentration monitoring systems that are embedded with carbon dioxide (CO_2) sensors are essential for actions against global

warming for monitoring the greenhouse effect. Monitoring of gas concentrations requires gas sensors with long-term stability and sufficient accuracy in their measurements. To keep the costs low, one often includes components that have unknown dependencies on external factors that can lead to a deduction in the accuracy of the sensor measurement, which is also known as drift. There are several typical working principles for the gas sensors, such as metal-oxide semiconductors, electrochemical cells, or optical gas sensing. In this article, we focus on the optical nondispersive infrared (NDIR) gas sensor, which is one of the most common optical gas sensors. Electrochemical sensors can be sensitive and relatively specific, but the lifespan of this type of sensor is usually short due to the sensor deterioration upon exposure to gases. Furthermore, the cross-sensitivity of other gases leads to the interference from other gases. The semiconductor sensors are simple, inexpensive, and scalable, but they have low specificity and can deteriorate as nontarget gases may be absorbed by the oxide surface, which leads to the limited lifetime. Compared to these sensors, the NDIR sensors provides high specificity, low life-cycle cost, minimal drift, and stable long-term operation [5]. However, previous works, such as [6]–[8], have recognized that NDIR sensors are sensitive to the variations of ambient temperature, atmospheric pressure, humidity, and some other environmental factors, which cannot always be compensated for.¹ Due to this, regular calibration is needed for long-term accuracy of the sensors.

Today, the state-of-the-art of infrared gas sensor self-calibration is the well-established ABC technology (automatic baseline correction) where the sensor is calibrated to a fixed value, that is, assumed to be the fresh air gas concentration [10]. However, this method does not work well in megacities where the sensors never get exposed to fresh air. Furthermore, the method cannot be used in the rapidly growing market of environmental sensors where the baseline, or fresh air gas concentration, is the measurement of primary interest. Thus, designing smart, more robust, and networked calibration algorithms, which can be widely applied in different environments, becomes more and more important.

In recent years, there has been a growing interest in studying the data-driven-based sensor calibration algorithms. Based on the analysis of the data about a specific system,

Manuscript received 23 December 2021; accepted 7 January 2022. Date of publication 20 January 2022; date of current version 25 July 2022. This work was supported by the European Union's Horizon 2020 Research and Innovation Programme under Grant 825272 (ULISSES). (*Corresponding author: Yang You.*)

Yang You, Hao Chen, and Tobias J. Oechtering are with the School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology, 100 44 Stockholm, Sweden (e-mail: youy@kth.se; haoch@kth.se; oech@kth.se).

Kai You is with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: sdlyyk@sjtu.edu.cn).

Digital Object Identifier 10.1109/IIOT.2022.3144934

2327-4662 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

¹In our previous work [9], the impact of ambient temperature on a certain type of NDIR CO_2 sensor produced by the company Senseair has also been experimentally studied.

the data-driven modeling method aims to find relationships between the system state variables (input and output) without explicit knowledge of the physical behavior of the system [11]. For instance, in [12], the concept of a data-driven model is proposed for calibrating the motion or position capture of the robot. In [13], different data-driven models are built for laser scanner calibration. Several other research works such as [14]–[16] have applied data-driven modeling methods for calibration of different sensors or even sensor networks. Meanwhile, there is a growing interest in machine learning-based sensor calibration. Machine learning approaches can provide increasing levels of automation and improved accuracy by discovering and exploiting dependencies in the (training) data. Recent research on data-driven sensor self-calibration have combined and adopted machine learning approaches to help to build data-driven models. Examples of this type of research are artificial neural networks for laser scanner calibration [13], decision tree-based learning for the calibration of airborne particulate sensors [17], and support vector machine-based pressure sensor calibration model [18]. In this article, we follow this idea of exploiting the concept of data-driven modeling and approaches of machine learning to enhance the sensor calibration mechanism. In more details, we utilize the hidden Markov model (HMM) [19] as a statistical learning tool to build a data-driven self-calibration system for NDIR gas sensors.

Next, we provide a sensing model and the corresponding drift analysis, where the measurement error happens due to the imperfect compensation for the variation of the behavior of the sensor components. Note that the concepts provided in this article are not restricted to NDIR gas sensors only, and our results can also be applied to other sensors that fit this more general sensing and drift model. Generally speaking, the basic working mechanism of a sensor is to convert a specific physical quantity to an electrical signal. Denoting the intensity of the converted electrical signal as E , consider the following general sensor measurement model:

$$O = f(r, E, F) \quad (1)$$

where O is the sensor output, F describes certain environmental factors, and r denotes a calibration parameter. The function $f(\cdot)$ describes the physical model of the sensor and maps the pair (r, E, F) to a certain sensor output O .

We call the calibration parameter that perfectly compensates the imperfect mapping f as *true calibration parameter*, which is denoted as x in the following. The true calibration parameter can be obtained by operating the sensor under reference conditions, i.e., we can solve (1) for x given the true values for O , E , and F . However, true values are usually unavailable while the sensor is operating. The goal is therefore to develop a data-driven calibration procedure to learning the true calibration parameter to enhance the performance of the sensors. To this end, we first propose a stochastic model of the true calibration parameter so that we later can estimate the true calibration parameter at any time instant given noisy observations of the sensor and environment factors using this stochastic model.

To address the above issues and challenges, we first propose to use an HMM as a joint stochastic model between observed

sensor measurements sequence, observed environmental factor sequences, and the sequence of the true calibration parameter. The calibration of a single sensor is then performed using statistical inference tools from the standard HMM framework. In the context of an IoT-based gas sensor system, the belief function of the true calibration parameter, i.e., the posterior distribution of the true calibration parameter, is derived for each single sensor according to its own joint probabilistic model. Exploiting that sensors can exchange their belief functions on the true value of the measured physical quantity (translated from the belief function of the true calibration parameter), we propose a Dempster–Shafer (DS) rule-based belief function fusion framework, which results in a networked calibration algorithm that can be performed at fusion center in the cloud. As an extension, we further formulate our calibration problem into a stochastic optimal control problem. We design an IoT-based gas sensor system, where the deep reinforcement learning (DRL) agent in the cloud can efficiently solve the stochastic optimal control problem and provide the calibration results.

The main contributions of this article lie in the following aspects.

- 1) We propose an HMM-based data-driven calibration method for the type of sensors that fit the sensing and drift model as described above. In more details, we first build the joint stochastic model between observed sensor measurements sequence, observed environmental factor sequences, and the sequence of true calibration parameter. The stochastic model is designed to capture the dependency between the behavior of the sensor components and the environmental factors. The calibration of a single sensor is achieved by inferring the true underlying calibration parameter at any time instance by using statistical inference tools from the HMM framework.
- 2) In the context of an IoT-based gas sensor system, assuming that all sensors can exchange the belief functions on the true value of the measured physical quantity (translated from the belief function of the true calibration parameter derived by HMM for single sensor), we propose a general DS rule-based belief function fusion framework for the multisensor fusion problem for the calibration of sensors with the sensing and drift model as described above. We next propose a modified weighted averaging approach to deal with the case where belief functions highly conflict with each other. Unlike the existing works, we propose to use the Wasserstein distance as a measure of distance between belief functions so that our approach becomes also applicable for the case where the belief functions are just simple Bayesian probability measures. As an extension, we propose a partially observed Markov decision process (POMDP) reformulation of our calibration problem. We design a DRL-based algorithm, which can solve the corresponding control problem computationally efficiently and can provide stationary calibration strategies over an infinite time horizon. In order to test the performance of our algorithm, we also built a real data-based simulation environment for the whole IoT-based NDIR

sensor system, which allows the interaction between the sensors, DRL agent, and other devices. With some modifications, this environment can be generalized and utilized for later research on the DRL-based calibration algorithms for the sensors with similar sensing and drift model.

The remainder of this article is organized as follows. The related works are introduced in Section II. The working mechanism of the NDIR gas sensor is introduced in Section III. The HMM-based single sensor calibration method is proposed in Section IV, while belief function fusion-based and DRL-based networked calibration methods are proposed in Section V. In Section VI, the performance of the proposed algorithms is illustrated and compared, where the results are analyzed and discussed. Finally, Section VII concludes this article.

II. RELATED WORKS

A. Multisensor Data Fusion

To reduce inaccuracy and uncertainty of single sensor measurements, the IoT-based gas concentration monitoring system with multiple sensors is often deployed [2]. In such an IoT system scenario, improved calibration can be achieved by multisensor data fusion, where information from different sensors are combined in the cloud to provide an overall more accurate gas concentration measurement. To deal with the uncertainty caused by the imperfection of the data, different traditional Bayesian probability theory-based approaches have been proposed as data fusion techniques. As a special case of the Bayesian filters, the Kalman filter and its variants have been widely used for sensor fusion in many existing works, such as [20]–[22].

As a generalization to the traditional Bayesian probability theory, the belief function theory (also known as the DS theory) [23] provides a particularly convenient theoretical framework for uncertainty modeling and propagation in the combination of partially reliable information. It has been widely applied to multisensor data fusion, such as [24]–[26]. However, the traditional DS theory will result in unreasonable fused beliefs when the belief functions provided by different sensors highly conflict with each other. To mitigate this issue, preprocessing methods of the original belief functions have been proposed [27]. Murphy [28] proposed to incorporate an average belief into the DS combining rule. However, this simple average approach assigns equal weight to each body of evidence and does not consider the relationship among the evidence collected from multiple sensors, which is often unreasonable in real application. For those cases, different weighted average approaches [29]–[31] have been proposed to manage the conflicts between belief functions and are shown to have good performance for the specific application. In this article, based on the stochastic model we built for the single sensor using HMM, we further propose a general belief function fusion framework for our proposed multisensor fusion-based NDIR gas sensor calibration problem. Then, we propose a modified weighted average approach to deal with the case where belief functions highly conflict with each other. Different from the aforementioned works [29]–[31], which

utilize the Jousselme distance [32] or the modified Jousselme distance as a measure of the distance between the belief functions, we propose to use the Wasserstein distance [33] as a measure of distance between the belief functions. This is done since the Jousselme distance is not applicable for the case where the belief functions are just simple Bayesian probability measures.

B. Deep Reinforcement Learning

The aforementioned multisensor fusion only provides an instantaneous solution for the NDIR gas calibration problem, without considering the overall long-term performance. It is however easy to be implemented since it does not require the statistical relation between sensors, i.e., joint statistics. It will work well in scenarios where the majority of the belief functions provide reliable and correct evidence. Moreover, the fusion result provided by this approach can be used as an approximation of the joint statistics of the system. With this approximated joint statistics, we next formulate the calibration problem as a Markov decision process (MDP) problem [34], where the long-term stable calibration results are derived in a sequential manner. Reinforcement learning provides model-free methods to solve such optimal control problems of dynamic systems without requiring the knowledge of the stochastic properties of the underlying model. Due to this convenience, reinforcement learning methods have been widely applied in sensor calibration problems during the recent years. Cao *et al.* [35] proposed a *Q*-learning-based Gaussian mixture model (GMM) algorithm for vehicle tracking via multisensor fusion. Meanwhile, a SARSA [36]-based reinforcement learning sensor fusion algorithm is proposed for autonomous robot navigation in [37]. When combined with deep learning, the more powerful DRL methods can deal with the curse-of-dimensionality problem by approximating the value functions as well as policy functions using deep neural networks. In this article, we reformulate our calibration problem as a stochastic optimal control problem and propose to use a deep *Q*-network (DQN) to solve it.

III. NDIR SENSOR MECHANISM

The general operation principle of an NDIR sensor is illustrated Fig. 1. When the sensor starts working, the broadband infrared light will be directed through the gas chamber filled by gas from the external environment toward the detector. A general wavelength range of interests for the NDIR sensing is 3–20 μm . However, NDIR sensing does not require light sources and detectors with narrow spectral range of operation. As long as the source and detector cover the wavelength range of interest, i.e., the range containing the target gas absorption band of interest, an NDIR measurement can be implemented. The IR light with different wavelengths is absorbed by gas molecules in the volume probed by the light. A bandpass optical filter is used at the detector side, which eliminates all light except the light with the wavelength that the target gas molecules can absorb, which provides the specificity to the target gas against other gases that might be present in the volume probed by the IR light. This principle provides the basis for

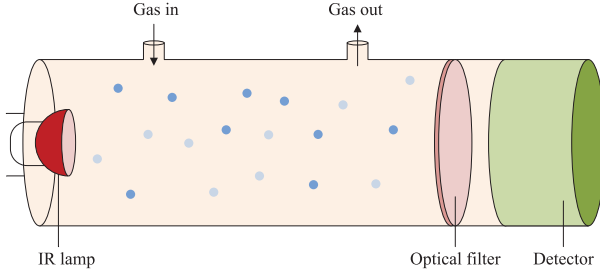


Fig. 1. Generic working mechanism of NDIR sensor. There are three essential components for NDIR sensing: IR light source, optical filter, and IR light detector. A general wavelength range of interests for both light source and the detector of the NDIR sensing is 3–20 μm . Light from the infrared lamp is absorbed by gas particles in the volume probed by the light. The intensity of remaining light is measured, which provides information about the gas concentration.

specifying the sensing model (1). In the rest of this work, we focus on the CO_2 NDIR sensors for which the temperature dependency is the most dominant effect on the behavior of the sensor components [7]. Accordingly, the CO_2 gas sensor model can be described as

$$y = g(r, i, c) \quad (2)$$

where c denotes the temperature, i is the current received by the detector, and r stands for the calibration parameter of the NDIR CO_2 sensor. The function $g(\cdot)$ describes the NDIR CO_2 sensor physical model based on the Beer–Lambert law that maps (r, i, c) onto the CO_2 concentration level y . However, the mapping g does not perfectly capture the dependency between the behavior of the infrared light and the temperature. As previously proposed, we want to adjust the calibration parameter r to its true value x according to the drift of the current i to compensate for the imperfections of the mapping g . To model the remaining uncertainties of the above quantities during the operating period of the sensor, we define random variables for these corresponding quantities. Specifically, we use C , I , Y , and X to denote the corresponding random variables of the environmental temperature, the current received by the detector, the CO_2 measurement, and the true calibration parameter.

Notation: In the following, we denote a random variable by a capital letter, its realization by the corresponding lowercase letter, and its alphabet by the corresponding calligraphic letter. We further denote a random sequence $(X_t, \dots, X_{t+k})$ and its realization $(x_t, \dots, x_{t+k})$ by $X_{t:t+k}$ and $x_{t:t+k}$, respectively.

IV. HIDDEN MARKOV MODEL-BASED STOCHASTIC MODELING OF INDIVIDUAL NDIR SENSOR DRIFT PROCESS

In this section, we propose to use an HMM to model the probabilistic relationship between the true calibration parameter and the observations of an NDIR sensor. To characterize the HMM, we first need to learn the model parameters from the data using either supervised or unsupervised learning methods.

A. Probabilistic State Space Model

In this section, as well as the rest of this article, to reduce the complexity of our proposed problem, we restrict all the

relevant variables in our proposed model to lie in finite discrete spaces. We also assume that the environmental temperature at each time instance can be observed without any noise. As we mentioned before, the unobservable true calibration parameter X is adjusted according to the drift of I caused by the temperature change. Thus, given fixed values for current i and temperature c , there is a one-to-one mapping relationship between the true CO_2 concentration level and the true calibration parameter x given by the deterministic mapping $g(\cdot)$. Since the true CO_2 level (or true calibration parameter) is not directly observable, we model the true calibration parameter as the hidden state that evolves with time. We use $\{X_t\}_{t=1}^T$ to denote the discrete-time stochastic process that describes the hidden states. While the sensor is operating using the calibration parameter r , we observe the noisy sensor measurement Y of the true CO_2 concentration level. Thus, we let $\{Y_t\}_{t=1}^T$ denote the discrete-time stochastic process of the observations of our proposed HMM. In this work, we focus on compensating the temperature dependency of the drift process. To this end, we let $\{C_t\}_{t=1}^T$ denote the discrete-time stochastic process of the temperature, that is, fully observed. The random variable $\Delta_t = C_t - C_{t-1}$ denotes the change of the temperature between two consecutive sampling instances. The experimental studies with the actual NDIR sensor data showed that it is sufficient to include only the temperature change to characterize the temperature dependency of the transition of the true calibration parameter, i.e., $P_{X_{t+1}|X_t^*, C_t^{t+1}} = P_{X_{t+1}|X_t^*, \Delta_t^{t+1}}$. At time step t , we further assume that the transition of the true calibration parameter only depends on the current state X_t and the temperature change from time t to time $t+1$, i.e.,

$$P_{X_{t+1}|X_t^*, \Delta_t^{t+1}} = P_{X_{t+1}|X_t, \Delta_{t+1}}. \quad (3)$$

We also assume the emission of the current observation Y_t only depends on the current state X_t . In this case, we end up with an HMM, which is fully characterized by the following distributions.

- 1) *Time-Invariant Transition Probability:* $X_{t+1} \sim P_{X_{t+1}|X_t, \Delta_{t+1}}$.
- 2) *Time-Invariant Emission Probability:* $Y_t \sim P_{Y_t|X_t}$.
- 3) *Prior Distribution:* $X_0 \sim P_{X_0}$.

We further assume that the true calibration parameter takes values from a finite integer set. Therefore, the hidden state corresponds to the quantized true calibration parameter. Assuming there are N such quantized values in total, hidden state X_t takes values from the set $\mathcal{X} = \{x_i\}_{i=1}^N \subset \mathbb{Z}$. Similarly, we assume that the CO_2 measurements can only take integer values within a certain range. In this case, given there are M possible CO_2 measurements in total, the random variable Y_t takes values from the set $\mathcal{Y} = \{y_j\}_{j=1}^M \subset \mathbb{Z}$. Likewise, the quantized temperature change takes values from the finite set $\bar{\Delta} = \{\delta_l\}_{l=1}^L \subset \mathbb{R}$. For compactness, we denote the transition probability $P_{X_{t+1}|X_t, \Delta_{t+1}}(x_{t+1}|x_t, \delta_l)$ as $A_{ii'|\ell}$. The set $A = \{A_{ii'|\ell}\}_{i,i',\ell}$ thus describes the transition behavior of the true calibration parameters given different temperature change values. Likewise, the emission probabilities are denoted by $B_i(j) = P_{Y_t|X_t}(y_j|x_i)$, $B = \{B_i(j)\}_{i,j}$, which is the likelihood of CO_2 measurement at time t given different X_t . Finally, we

define $\rho_i = P_{X_0}(x_i)$, $\rho = \{\rho_i\}_i$, as the initial prior distribution of the hidden state.

Remark 1: Given the above definitions, the parameters $\{\pi, A, B\}$ fully characterize a statistical model of the drift process of a single sensor.

The above HMM parameters can be learned via either a supervised approach or an unsupervised approach. In this article, since we have labeled training data set, we only choose to use the supervised approach, which provides good performance with much lower complexity than the unsupervised approach. In the following, we briefly outline the supervised approach on learning these HMM parameters. For the details of the unsupervised approach and its corresponding experiments, see our previous work [38] for more details.

B. Supervised Learning of HMM

Suppose we have a data set containing the CO_2 measurements of a sensor to be calibrated and a reference sensor. The CO_2 measurements of the reference sensor can be regarded as true CO_2 concentration levels. Thus, the corresponding true calibration parameter can be computed with sufficient accuracy by using the CO_2 measurements of the reference sensor. We can then approximate the parameters $\{\rho, A, B\}$ via a simple supervised learning approach by calculating the relative frequencies of the state transitions and emissions.

Let D be the total number of data points. The transition probability can be estimated by calculating the following relative frequency:

$$A_{i'i|l}^{(D)} = P_{X_{t+1}|X_t, \Delta_{t+1}}^{(D)}(x_{i'}|x_i, \delta_l) = \frac{\text{count}(X_{t+1} = x_{i'}, X_t = x_i, \Delta_{t+1} = \delta_l)}{\text{count}(X_t = x_i, \Delta_{t+1} = \delta_l)}. \quad (4)$$

Similarly, the emission probability can be estimated by the relative frequency

$$B_{ij}^{(D)} = P_{Y_t|X_t}^{(D)}(y_j|x_i) = \frac{\text{count}(Y_t = y_j, X_t = x_i)}{\text{count}(X_t = x_i)}. \quad (5)$$

Finally, the prior distribution of the hidden states can be approximated by the relative frequency

$$\rho_i^{(D)} = P_{X_t}^{(D)}(x_i) = \frac{\text{count}(X_t = x_i)}{N}. \quad (6)$$

V. JOINT CALIBRATION FOR NDIR SENSOR NETWORK IN IOT-BASED GAS MONITORING SYSTEM

The previous HMM-based self-calibration method only works for a single sensor. It will perform poorly when the stochastic calibration model does not characterize the drift behavior accurately or when the model becomes out-of-date. The uncertainty of the single sensor measurement and calibration can be reduced by deploying multiple sensors to monitor the gas concentration. Thus, in this section, we consider the networked calibration procedure in an IoT-based gas monitoring system, where a joint calibration is performed on multiple sensors. In the following, we first introduce the infrastructure of our IoT-based gas monitoring system. We further propose two different joint calibration algorithms for the NDIR sensor network in this IoT system.

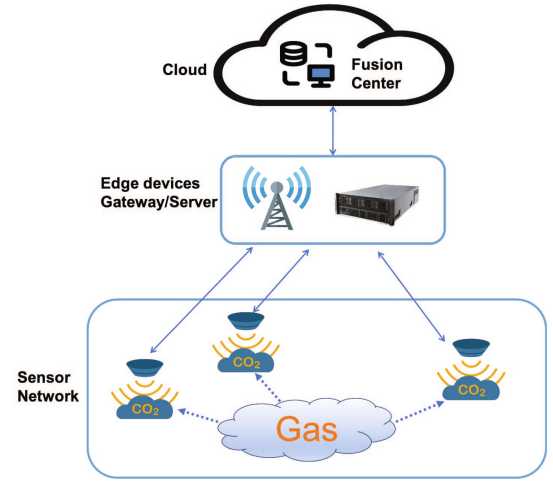


Fig. 2. Working mechanism of the IoT-based NDIR CO_2 sensor system with multiple locally distributed sensors, edge devices, and one fusion center in the cloud.

A. System Overview

Consider an IoT-based NDIR CO_2 sensor system with multiple NDIR sensors that measure the time-varying CO_2 concentration level in the same environment over a certain time horizon, i.e., the sensors are measuring the same CO_2 level at each time instant. By using the HMM framework in Section IV-A, each sensor can compute its own posterior distribution of the current true CO_2 level given historical measurements, i.e., belief functions of the current true CO_2 level, at the local computing node. Since the sensors are unaware of the true CO_2 level, to infer on the current true CO_2 level, the sensors will send their belief functions on the true CO_2 level to a fusion center in the cloud through the gateway at the IoT edge.² After running certain designed joint calibration algorithms, the cloud fusion center will send the calibration results back to all the local sensors through the gateway. The system model described above is depicted in Fig. 2.

In order to store the data sent from the sensors and compute the calibration results for all sensors, the cloud fusion center should be composed by certain computing nodes and storage nodes. The amount of needed computational resources depends on the choice of the algorithms as well as the dynamic of the environment. There always be tradeoff between the amount of required computational resources and the calibration accuracy. If we can assume that the environment is slowly varying compared to the sensing interval, then there is sufficient time for the cloud fusion center to do the computation before the true CO_2 level changes. As long as the fusion result is computed and sent back to the sensors before the true CO_2 level

²Note that all the sensors should transmit belief functions that correspond to measurements done during the same period while the environmental conditions remain sufficiently unchanged. Otherwise, the calibration result will be bad since the belief functions on true CO_2 levels of different environments will be fused. This requires that the sensors synchronously take the measurement during the same period while the environmental conditions remain sufficiently unchanged. In case the environment is slowly varying, the condition for the synchronous sensing will be relaxed since the sensors can take the measurement at any time in a relatively long period while the environment remains unchanged.

changes, it would be valid for the current period and can be utilized as reference to calibrate all sensors. In case the environment varies fast, the cloud needs to do fast computation and give the sensors a quick response. However, the fast computation can still be achieved by limited computational resources with lower calibration accuracy, e.g., have more rough quantization for the true CO_2 levels to make the belief function vectors shorter. Otherwise, since there will be a delay caused by the limited computational resources, sensors need to store intermediate CO_2 and temperature measurements during the delay period, do the calibration at a later stage when they receive the calibration result and then replay the belief function evolution.

B. On Deriving the Belief Function

Assume we have $\bar{N}$ sensors in total. Let $z_t \in \mathcal{Z}$ denote the true CO_2 level of the environment at time t . As before, we quantize the true CO_2 level into L integer values. Thus, $\mathcal{Z} = \{z_l\}_{l=1}^L \subset \mathbb{Z}$ denotes the set of all possible quantized true CO_2 levels. Similarly, we define $i_t \in \mathcal{I}$ as the current value at time t , where the set $\mathcal{I}$ contains $\bar{L}$ integer values within a certain range, i.e., $\mathcal{I} = \{i_t^l\} \subset \mathbb{Z}$.

Given the above notations, we provide the following definitions of the involved belief functions.

- 1) At time t , the posterior distribution of the true calibration parameter (*belief function of true calibration parameter*) of sensor n is defined as $P(x_t^n | y_{1:t-1}^n, c_{1:t})$, where x_t^n denotes the true calibration parameter of sensor n at time t , $y_{1:t-1}^n$ denotes the historical CO_2 measurement from the first time step until time $t-1$ of sensor n , and $c_{1:t}$ is the historical environmental temperature sequence from the first time step until time t .
- 2) At time t , the posterior distribution of the true CO_2 level (*belief function of true CO_2 level*) of sensor n specified by the current value i_t^n is defined as $P_{i_t^n}(z_t | y_{1:t-1}^n, c_{1:t})$.
- 3) At time t , the joint posterior distribution of the true CO_2 level (*belief function of true CO_2 level*) of all sensors at time t is defined as $P(z_t | y_{1:t-1}^{1:\bar{N}}, c_{1:t})$, where $y_{1:t-1}^{1:\bar{N}}$ denotes the historical CO_2 measurement sequence of all $\bar{N}$ sensors.

Utilizing the HMM parameters for single sensor and the Bayesian rule, the belief function of the true calibration parameter can be updated according to the following lemma.

Lemma 1: The belief function $P(x_t^n | y_{1:t-1}^n, c_{1:t})$ can be updated according to the following rule:

$$P(x_t^n | y_{1:t}^n, c_{1:t}) = \frac{P(x_t^n | y_{1:t-1}^n, c_{1:t})P(y_t^n | x_t^n)}{\sum_{x_t^n \in \mathcal{X}} P(x_t^n | y_{1:t-1}^n, c_{1:t})P(y_t^n | x_t^n)} \quad \forall x_t^n \in \mathcal{X}. \quad (7)$$

$$P(x_{t+1}^n | y_{1:t}^n, c_{1:t+1}) = \sum_{x_t^n \in \mathcal{X}} P(x_t^n | y_{1:t}^n, c_{1:t})P(x_{t+1}^n | x_t^n, y_{1:t}^n, c_{1:t+1}) \quad \forall x_{t+1}^n \in \mathcal{X}. \quad (8)$$

Proof: According to the definition of the emission probability in our HMM, we then have the Markov chain $y_t^n - x_t^n - (y_{1:t-1}^n, c_{1:t})$, which leads to $P(y_t^n | x_t^n) = P(y_t^n | x_t^n, y_{1:t-1}^n, c_{1:t})$.

Plugging $P(y_t^n | x_t^n) = P(y_t^n | x_t^n, y_{1:t-1}^n, c_{1:t})$ into (7), the equation is easily to be verified. Besides, (8) holds since we have $P(x_{t+1}^n | x_t^n, y_{1:t}^n, c_{1:t+1}) = P(x_{t+1}^n | x_t^n, \delta_{1:t+1}) = P(x_{t+1}^n | x_t^n, \delta_{t+1})$ according to (3), and $P(x_t^n | y_{1:t}^n, c_{1:t}) = P(x_t^n | y_{1:t}^n, c_{1:t+1})$ according to the Markov chain $x_t^n - (y_{1:t}^n, c_{1:t}) - c_{t+1}$. Thus, given the HMM parameters $\{\pi^n, A^n, B^n\}$ for sensors n , the belief function $P(x_t^n | y_{1:t-1}^n, c_{1:t})$ at any time step t can be calculated according to (7) and (8) by initializing the belief function as the prior distribution π^n for sensor n . ■

Remark 2: At time t , given fixed i_t^n and $c_t \quad \forall z_t \in \mathcal{Z}$, if there is any $\forall x_t^n \in \mathcal{X}$ that satisfies $z_t = g(x_t^n, i_t^n, c_t)$, the belief function of true CO_2 level $P_{i_t^n}(z_t | y_{1:t-1}^n, c_{1:t})$ is then calculated by

$$P_{i_t^n}(z_t | y_{1:t-1}^n, c_{1:t}) = P(x_t^n = h(z_t, i_t^n, c_t) | y_{1:t-1}^n, c_{1:t}) \quad (9)$$

where the function $h(\cdot)$ is the inverse of function $g(\cdot)$ such that $g(h(z_t, i_t^n, c_t), i_t^n, c_t) = z_t \quad \forall i_t^n, c_t$, which is a one-to-one mapping between z_t and x_t^n when i_t^n and c_t are fixed. Otherwise, $P_{i_t^n}(z_t | y_{1:t-1}^n, c_{1:t}) = 0$.

With the belief function of true CO_2 level calculated according to Remark 2, we next propose a method to fuse these belief functions. For notation simplicity, we use $P_n(z_t)$ to denote the belief function of the true CO_2 level for sensor n at time t and $P(z_t)$ to denote the joint belief function of the true CO_2 level for all sensors at time t in the following part of this section.

C. Belief Function Fusion-Based Self-Calibration for NDIR Sensor Network

For this calibration scheme, the belief function fusion approach will be performed at the cloud fusion center to get a fused belief function. The fused belief function can finally be adopted as updated belief by all sensors (calibration). Assume there are $\bar{N}$ sensors in total and the belief functions provided by all sensors are reliable. Our objective then is to design a fusion rule to combine the beliefs provided by all $\bar{N}$ sensors. The DS rule [23] provides a general framework for such a reasoning problem with uncertain information or partial information. Let $n, m \in \{1, 2, \dots, \bar{N}\}$ be the indices of two sensors. We first consider the most simple two sensor fusion case, where we wish to combine the belief functions provided by sensor n and sensor m . Applying the DS combination rule with singleton sets results in the following fusion rule:

$$(P_n \oplus P_m)(z_l) = \frac{1}{1-F} P_n(z_l) P_m(z_l) \quad \forall z_l \in \mathcal{Z} \quad (10)$$

where $F = \sum_{z_{l_1}, z_{l_2} \in \mathcal{Z}} P_n(z_{l_1}) P_m(z_{l_2})$ denotes the normalizing factor. The operator $\oplus$ denotes the combination operator via DS rule.

For the N sensors fusion case, the N -fold extension of the DS rule with singleton sets results in

$$P(z_l) = (P_1 \oplus P_2 \oplus \dots \oplus P_{\bar{N}})(z_l) = \frac{1}{F'} P_1(z_l) P_2(z_l) \dots P_{\bar{N}}(z_l) \quad \forall z_l \in \mathcal{Z} \quad (11)$$

where the normalizing factor is $F' = \sum_{z_l \in \mathcal{Z}} P_1(z_l) P_2(z_l) \dots P_{\bar{N}}(z_l)$. The fused belief $P(z)$ can be then further used by all the sensors.

The DS rule will result in unreasonable fused beliefs when the belief functions provided by different sensors highly conflict with each other, e.g., the DS rule will fail when some sensors hold highly different belief functions compared to the other sensors in particular when the intersection of the support sets of the beliefs is empty. To address this issue, we utilize the weighted average approach with a weighting of each belief function based on its similarity with other belief functions. To measure the similarity, we propose to use the Wasserstein distance as a distance measure between the belief functions. The Wasserstein distance is suitable for the case where the underlying support sets of two probability measures are very different.

Let X and Y be two random variables with probability distribution P_X and P_Y . The Wasserstein distance W_2 between P_X and P_Y is defined as

$$W_2(P_X, P_Y) = \sqrt{\min_{P_{XY}: \sum_x P_{XY} = P_Y, \sum_y P_{XY} = P_X} \sum_{x \in \mathcal{X}, y \in \mathcal{Y}} |x - y|^2 P_{XY}(x, y)}. \quad (12)$$

The distance between the belief functions of sensor n and sensor m is then given by $W_2(P_n(z), P_m(z)) \forall n, m \in \{1, 2, \dots, \bar{N}\}$. In the following, we follow the setup as proposed in [29]. In more detail, after getting the distance between each pair of sensors, we normalize all distances into the interval $[0, 1]$ as follows:

$$\hat{W}_2(P_n(z), P_m(z)) = \frac{2 \times W_2(P_n(z), P_m(z))}{\sum_n \sum_m W_2(P_n(z), P_m(z))}. \quad (13)$$

With the normalized distance measure provided above, we define the similarity measure between belief functions $P_i(z)$ and $P_j(z)$ as

$$S(P_n(z), P_m(z)) = 1 - \hat{W}_2(P_n(z), P_m(z)). \quad (14)$$

As in [29], we next define the support degree of a given belief function $P_i(z)$ as

$$\text{Supp}(P_n(z)) = \sum_{m=1, m \neq n}^{\bar{N}} S(P_n(z), P_m(z)) \quad (15)$$

which characterizes the relative importance of the belief function P_i . The weighting factor of belief function P_i is then obtained by the following normalization:

$$\alpha_n = \frac{\text{Supp}(P_n(z))}{\sum_{n=1}^{\bar{N}} \text{Supp}(P_n(z))}. \quad (16)$$

The weighted average of all $\bar{N}$ belief functions can be expressed as $\hat{P}(z) = \sum_{i=1}^{\bar{N}} \alpha_n P_n(z)$. The Wasserstein metric-based fused belief function is finally obtained by using the DS rule to combine $\hat{P}(z)$ for $N - 1$ times

$$P(z) = (\hat{P} \oplus \hat{P} \oplus \dots \oplus \hat{P})(z) \quad (17)$$

where we apply “ $\oplus$ ” for $N - 1$ times as proposed in [29].

Remark 3: The probability mass function $P(z_t)$ derived by the above procedure, which is the fusion result of the belief functions $P_t^n(z_t|y_{1:t-1}^n, c_{1:t}) \forall n \in \{1, 2, \dots, \bar{N}\}$, is used as an approximation of the joint belief function $P(z_t|y_{1:t-1}^{1:\bar{N}}, c_{1:t})$

since it is difficult to obtain a joint statistics of the whole system. We denote this approximated belief function as $P_t^{1:\bar{N}}(z_t|y_{1:t-1}^{1:\bar{N}}, c_{1:t})$, which is further used in the next section, where $i_t^{1:\bar{N}}$ denotes the sequence of the current values for all $\bar{N}$ sensors at time t .

D. Sequential Self-Calibration for the NDIR Sensor Network via Deep Reinforcement Learning

The previous modified belief function fusion method only provides an instantaneous solution for the sensor calibration problem instead of considering the overall long-term performance. The advantage is that it is simple to be implemented since it does not require the statistical relation between sensors, i.e., joint statistics. It can work well in scenarios where the majority of the belief functions provide reliable and correct evidence. Moreover, the fusion result provided by this approach can be used as an approximation of the joint belief function $P(z_t|y_{1:t-1}^{1:\bar{N}}, c_{1:t})$, which is then used as the joint statistics of the system. This joint statistics allows us to propose the following reference belief function selection problem, which is further reformulated into a POMDP problem that with an objective considering the impact from both past and future.

1) POMDP Problem Formulation for Reference Belief Selection: Under the IoT-based CO_2 monitoring system model as illustrated in Section V-A, assume we have $\bar{N}$ sensors in total. At time step t , each sensor sends its belief function on the current true CO_2 level to the fusion center (decision maker) at the cloud. The fusion center will further decide on the best belief function (named reference belief function) $P_{b^*}(z_t)$, where b^* is the index of the sensor with the best belief. The reference belief $P_{b^*}(z_t)$ is then sent back to all sensors. Each sensor then will update its own belief function as the reference belief $P_{b^*}(z_t)$.

Considering the whole time horizon, the reference belief selection needs to be performed in a sequential manner such that the overall calibration scheme can achieve a long-term stable performance. In the following, we provide details on how to reformulate the sequential reference belief selection problem into a POMDP. According to the process described above, at time step t , the calibration action of the fusion center is defined as $b_t \in \mathcal{B} = \{1, 2, \dots, \bar{N}\}$. Thus, the control action decides on the index of the sensor, whose belief function on the true CO_2 level will be selected as reference belief. As the next step, we need to decide upon an objective function, i.e., the cost function (reward function) of the POMDP, which measures the calibration performance. Previous research, such as [39], has demonstrated the effectiveness of conditional entropy as such an objective function. Given the calibration action b_t , we propose to use the conditional entropy as the objective function as shown in (18) at the bottom of the next page. An intuitive explanation of the above objective is that we wish to select a reference sensor, whose individual statistical model of this sensor brings least uncertainty when guessing the current true CO_2 level given the observations of CO_2 measurements and temperature.

As shown in (18), the cost does not only depend on the current “true” state and the control action but also depends

on the historical sequences $(y_{1:t-1}^{1:\tilde{N}}, c_{1:t})$. In order to formulate it into a standard MDP, we introduce belief states, which will be used to replace the growing historical sequences by identifying an update rule. To this end, we identify the belief state $\theta_t \triangleq P(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t})$ as the state of our POMDP problem. However, as we mentioned before, the joint statistics $P(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t})$ is difficult to be obtained. Thus, we define $\tilde{\theta}_t \triangleq P_{i_t^{1:\tilde{N}}}(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t})$ as the approximation of the joint statistics $P(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t})$, where $P_{i_t^{1:\tilde{N}}}(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t})$ is the fusion result as defined in Remark 3. At each time step, on observing the sequences $(y_{1:t-1}^{1:\tilde{N}}, i_t^{1:\tilde{N}}, c_{1:t})$, the fusion center will obtain an approximation of the joint statistics, i.e., approximated belief state. Based on this approximation, the fusion center will further choose a calibration action according to the calibration policy. Accordingly, the calibration can be defined as $\pi_t \in \Pi_t$, where Π_t denotes the set of deterministic mappings from approximated belief state $\tilde{\theta}_t$ to a corresponding action b_t , i.e., $b_t = \pi_t(\tilde{\theta}_t)$. Correspondingly, the calibration policy over a T -time horizon is $\pi = \{\pi_t\}_{t=1}^T \in \Pi = \Pi_1 \times \Pi_2 \times \dots \times \Pi_T$.

With the elements for the POMDP defined above, considering long-term calibration performance, our overall objective is to find the calibration policy $\pi \in \Pi$ that minimizes the expected discounted total cost

$$L_T(\pi) = \mathbb{E}_\pi \left\{ \sum_{t=1}^T \gamma^t H(Z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}, Y_t^{b_t}) \right\} \quad (19)$$

where $\gamma \in [0, 1)$ is the discounted factor to determine the weight of the future cost. $\gamma = 0$ means that we only care about the immediate reward. When $\gamma < 1$, the costs in the future will cause less impact than the costs of the earlier time steps.

The optimal calibration policy is defined as

$$\pi^* = \arg \min_{\pi \in \Pi} L_T(\pi). \quad (20)$$

Given a state-action pair (θ_t, b_t) and the approximated belief state $\tilde{\theta}_t$, we provide the following lemma.

Lemma 2: The per-step cost defined in (18) can be written as a function of the state-action pair (θ_t, b_t) , and can be further approximated by substituting θ_t with $\tilde{\theta}_t$.

Proof: Note that there are two different probability terms included in (18), i.e., $P(y_t^{b_t} | y_{1:t-1}^{1:\tilde{N}}, c_{1:t})$ and $P(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}, y_t^{b_t})$. We further decompose these two terms as the following:

$$\begin{aligned} & P(y_t^{b_t} | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}) \\ &= \sum_{z_t} P(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}) P(y_t^{b_t} | z_t, y_{1:t-1}^{1:\tilde{N}}, c_{1:t}) \\ &\approx \sum_{z_t} P_{i_t^{1:\tilde{N}}}(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}) P(y_t^{b_t} | z_t, y_{1:t-1}^{1:\tilde{N}}, c_{1:t}) \end{aligned} \quad (21)$$

$$\begin{aligned} & P(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}, y_t^{b_t}) \\ &= \frac{P(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}) P(y_t^{b_t} | z_t, y_{1:t-1}^{1:\tilde{N}}, c_{1:t})}{P(y_t^{b_t} | y_{1:t-1}^{1:\tilde{N}}, c_{1:t})} \\ &\approx \frac{P_{i_t^{1:\tilde{N}}}(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}) P(y_t^{b_t} | z_t, y_{1:t-1}^{1:\tilde{N}}, c_{1:t})}{P(y_t^{b_t} | y_{1:t-1}^{1:\tilde{N}}, c_{1:t})}. \end{aligned} \quad (22)$$

At time step t , assume the reference sensor is selected as a sensor with index b_t denoted. Given fixed $i_t^{b_t}$ and $c_t \forall z_t \in \mathcal{Z}$, if there is any $x_t^n \in \mathcal{X}$ that satisfies $z_t = g(x_t^n, i_t^{b_t}, c_t)$, $P(y_t^{b_t} | z_t)$ can be then calculated according to the following:

$$P(y_t^{b_t} | z_t) = P(y_t^{b_t} | x_t^{b_t} = h(z_t, i_t^{b_t}, c_t)) \quad (23)$$

where $h(\cdot)$ is the inverse function of $g(\cdot)$ as defined in Remark 2. Otherwise, $P(y_t^{b_t} | z_t) = 0$. Note that $P(y_t^{b_t} | x_t^{b_t})$ is the emission probability of sensor b_t at time t , which is obtained via the supervised learning of the HMM parameters for each single sensor. The equation above indicates that $P(y_t^{b_t} | z_t)$ can be fully specified by the emission probability term of sensor b_t given control action b_t . Also, note that $P(y_t^{b_t} | z_t, y_{1:t-1}^{1:\tilde{N}}, c_{1:t}) = P(y_t^{b_t} | z_t)$ due to the conditional independent assumptions. In this case, the per-step cost in (18) thus can be fully written as a function of any state-action pair (θ_t, b_t) and thus, can be approximated by a function of the state-action pair $(\tilde{\theta}_t, b_t)$. ■

In this case, the per-step cost at time t is defined as the following function of state-action pair (θ_t, b_t) :

$$l_t(\theta_t, b_t) = H(Z_t | y_{1:t-1}^{1:\tilde{N}}, i_{1:t}^{1:\tilde{N}}, c_{1:t}, Y_t^{b_t}) \quad (24)$$

and we denote the approximated per-step cost as $l_t(\tilde{\theta}_t, b_t)$.

With the above model, we next identify the system dynamics, e.g., evolution of the approximated belief state $\tilde{\theta}_t$ given a certain control action b_t^* . The evolution is summarized into the following two steps.

- 1) At time step t , given assume the fusion center decides on the index of the reference sensor as b_t^* , the belief function $P_{i_t^{b_t^*}}(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t})$ is then sent back to all sensors as the reference belief function. After receiving the reference belief function, each sensor n will first update its own belief function $P(x_t^n | y_{1:t-1}^n, c_{1:t}) \forall n \in \{1, 2, \dots, \tilde{N}\}$ (belief function of current true calibration parameter). Given fixed i_t^n and $c_t \forall x_t^n \in \mathcal{X}$, if there is any $\forall z_t \in \mathcal{Z}$ that satisfies $z_t = g(x_t^n, i_t^n, c_t)$, $P(x_t^n | y_{1:t-1}^n, c_{1:t})$ can be then calculated according to the following equation:

$$\begin{aligned} & P_{\text{new}}(x_t^n | y_{1:t-1}^n, c_{1:t}) \\ &= P_{i_t^{b_t^*}}(z_t = g(x_t^n, i_t^n, c_t) | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}) \end{aligned} \quad (25)$$

where the function $g(\cdot)$ is the mapping function defined in (2). Otherwise, $P_{\text{new}}(x_t^n | y_{1:t-1}^n, c_{1:t}) = 0$. Thus, under

$$H(Z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}, Y_t^{b_t}) = \sum_{y_t^{b_t}} P(y_t^{b_t} | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}) \sum_{z_t} P(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}, y_t^{b_t}) \log P(z_t | y_{1:t-1}^{1:\tilde{N}}, c_{1:t}, y_t^{b_t}) \quad (18)$$

the control action b_t^* , each sensor n has now an updated belief function $P_{\text{new}}(x_t^n | y_{1:t-1}^n, c_{1:t})$.

- 2) At time step $t + 1$, for all sensors, the updated belief function $P_{\text{new}}(x_t^n | y_{1:t-1}^n, c_{1:t})$ will evolve to $P(x_{t+1}^n | y_{1:t}^n, c_{1:t+1})$ according to (7) and (8) in Section V-B. Each sensor then calculates $P_{t+1}^n(z_{t+1} | y_{1:t}^n, c_{1:t+1})$ using $P(x_{t+1}^n | y_{1:t}^n, c_{1:t+1})$ according to (9). The new belief state $\theta_{t+1} = P(z_{t+1} | y_{1:t}^{1:\bar{N}}, c_{1:t+1})$ is approximated by $P_{i: \bar{N}}(z_{t+1} | y_{1:t}^{1:\bar{N}}, c_{1:t+1})$, which is obtained by fusing $P_{t+1}^n(z_{t+1} | y_{1:t}^n, c_{1:t+1})$ of all sensors via the belief function fusion approach provided in Section V-C.

With the above formulation and derivation, according to the definition of POMDP in [34, pp. 150–151], we have the following theorem.

Theorem 1: The sequential self-calibration for the NDIR sensor network can be formulated as a POMDP problem such that: 1) the state at time t is given by a belief state $\theta_t \triangleq P(z_t | y_{1:t-1}^{1:\bar{N}}, c_{1:t})$; 2) the control action at time t is specified by $b_t \in \{1, 2, \dots, \bar{N}\}$; 3) the per-step expected cost corresponding to a state–action pair is given by (24); 4) given a control action b_t , the belief state will evolve according to the two-step process described above; and as a consequence we have that 5) the corresponding optimal calibration policy π^* can be derived by using the Bellman dynamic programming.

Remark 4: Due to difficulties in obtaining the joint statistics θ_t , we alternatively use $\tilde{\theta}_t \triangleq P_{i: \bar{N}}(z_t | y_{1:t-1}^{1:\bar{N}}, c_{1:t})$ as the approximated belief state of our POMDP problem. Accordingly, all the calculations involved in the Bellman dynamic programming is done by using the state–action pair $(\tilde{\theta}_t, b_t)$ instead of using (θ_t, b_t) .

2) Backward Dynamic Programming Over Finite-Time Horizon: By applying the Bellman operator [34], the value function $V_t(\theta_t)$ is updated according to

$$V_t(\tilde{\theta}_t) = \min_{b_t \in \{1, 2, \dots, \bar{N}\}} l_t(\tilde{\theta}_t, b_t) + \gamma \mathbb{E}_{P(\tilde{\theta}_{t+1} | \tilde{\theta}_t, b_t)} [V_{t+1}(\tilde{\theta}_{t+1})] \quad (26)$$

where $\gamma \in (0, 1)$ is the discount factor and the term $\mathbb{E}_{P(\tilde{\theta}_{t+1} | \tilde{\theta}_t, b_t)} [V_{t+1}(\tilde{\theta}_{t+1})]$ denotes the expected cost-to-go. The optimal calibration policy $\pi_t^*(\tilde{\theta}_t)$ and the corresponding optimal calibration action $b_t^* = \pi_t^*(\tilde{\theta}_t)$ are given by the optimizer of (26). Furthermore, the minimum value of the expected discounted total cost is given by $V(\tilde{\theta}_1)$.

Unfortunately, the dynamic programming only provides a nonstationary solution for the above finite-time horizon POMDP problem. Also, note that the state space of our problem is continuous, i.e., we have infinite number of states. When dealing with high-dimensional problems, the computational complexity of dynamic programming grows rapidly. To avoid the curse of the dimensions and the growth of the complex system dynamics, we propose to use a deep Q -network as a computationally efficient method to solve the above POMDP problem. As an extension to the infinite time horizon, we provide stationary solutions for our reference belief selection

problem with the following objective:

$$L_T^{\text{inf}}(\pi) = \lim_{T \rightarrow \infty} \mathbb{E}_{\pi} \left\{ \sum_{t=1}^T \gamma^t H(Z_t | y_{1:t-1}^{1:\bar{N}}, c_{1:t}, Y_t^{b_t}) \right\}. \quad (27)$$

3) Deep Reinforcement Learning-Based Reference Belief Function Selection: In this section, we consider the infinite time horizon case. Since the system statistics and dynamics are assumed to be time invariant, we use $(s, a) \in \mathcal{S} \times \mathcal{A}$ to denote the belief state and action pair $(\tilde{\theta}_t, b_t)$, $l(s, a)$, and $P(s' | s, a)$ as the cost $l_t(\tilde{\theta}_t, b_t)$ and transition probability to state s' from the state–action pair (s, a) , which corresponds to $P(\tilde{\theta}_{t+1} | \tilde{\theta}_t, b_t)$. Under the infinite time horizon, the optimal Bellman equation of our reference belief selection problem becomes

$$V(s) = \min_{a \in \{1, 2, \dots, \bar{N}\}} l(s, a) + \gamma \mathbb{E}_{P(s' | s, a)} [V(s')]. \quad (28)$$

The optimal policy $\pi^* = (\pi, \pi, \dots)$ is stationary and is defined by

$$\pi(s) = \arg \min_{a \in \{1, 2, \dots, \bar{N}\}} [l(s, a) + \gamma \mathbb{E}_{P(s' | s, a)} V^*(s')] \quad \forall s. \quad (29)$$

In the following, we provide a brief outline to solve the above problem using DQN. More details on this method can be found in [40].

Given the optimal value function $V^*(s)$ in (29), we define the optimal Q -function $Q(s, a)$ as the optimal total expected cost starting from state s with action a as follows:

$$Q^*(s, a) = l(s, a) + \gamma \mathbb{E}_{P(s' | s, a)} V^*(s'). \quad (30)$$

Since $V^*(s) = \min_{a \in \{1, 2, \dots, \bar{N}\}} Q^*(s, a)$, we have

$$Q^*(s, a) = C(s, a) + \gamma \mathbb{E}_{P(s' | s, a)} \min_{b \in \{1, 2, \dots, \bar{N}\}} Q^*(s', b). \quad (31)$$

The traditional tabular Q -learning [36] can learn the optimal Q -function by utilizing the temporal difference between the new estimate and the old estimate. This is given by the following:

$$Q_{n+1}(s, a) = (1 - \beta) Q_n(s, a) + \beta \left[l(s, a) + \gamma \min_{b \in \{1, 2, \dots, \bar{N}\}} Q_n(s', b) \right] \quad (32)$$

where $\beta \in (0, 1]$ denotes the learning rate. The learning rate can be kept constant during the learning process. However, traditional tabular Q -learning cannot deal with the continuous state space problem. To this end, we propose to implement DQN to learn the function representation of the optimal Q -function. This function representation can be then applied to each state over the whole continuous state space.

In more detail, the Q -function $Q(s, a)$ is estimated by a function representation $Q(s, a, w)$, where w denotes the parameters of the deep neural network applied in DQN. To avoid the divergence of the algorithm, the DQN utilizes replay buffer D , which is filled by samples of the transition tuples, i.e., (s, a, r, s') . At each time step, the DQN agent will execute an action a selected by ϵ -greedy policy.³ At training step i , the

³In the reinforcement learning scenario, under an ϵ -greedy policy, the agent choose the best action with probability $1 - \epsilon$ and randomly choose an action with probability ϵ .

Algorithm 1: DQN-Based Reference Belief Function Selection and Self-Calibration for NDIR Gas Sensor Network

Input: HMM model parameters of each sensor $\{\rho, A, B\}$.

Initialization: DQN with initial Q -function $Q(s, a, w)$, learning rate α , experience replay buffer $\mathcal{S}$ with size S , mini-batch size N_D , and target model updating frequency F .

```

for each episode  $e = 1, 2, \dots, N^{epi}$  do
    Initialize the belief function of the true calibration parameter for
    each sensor as  $P_n(x_1)$  according to the prior distribution parameter
     $\rho$ .
    for each training step  $t = 1, 2, \dots, T$  do
        Each sensor observes its  $CO_2$  measurement  $y_t^n$ , current signal
         $i_t^n$ , temperature  $c_t$ , and calculates its belief function on true
         $CO_2$   $P_{t_t}^n(z_t | y_{1:t-1}^n, c_{1:t})$  using (9);
        Each sensor sends its belief function to the cloud;
        The cloud fusion center computes the approximated belief
        state  $\tilde{\theta}_t$  according to the fusion rule proposed in Section V-C,
        and selects a reference sensor  $b_t$  based on the  $\epsilon$ -greedy policy;
        The cloud fusion center executes action  $b_t$  by sending the
        reference belief function  $P_{t_t}^{b_t}(z_t | y_{1:t-1}^{b_t}, c_{1:t})$  back to all
        sensors, and observes the cost  $l_t(\tilde{\theta}_t, b_t)$ ;
        Each sensor updates belief function on true calibration
        parameter to  $P_{new}^n(x_t^n | y_{1:t-1}^n, c_{1:t})$  according to the reference
        belief function, which evolves to  $P(x_{t+1}^n | y_{1:t}^n, c_{1:t+1})$ 
        according to (7) and (8);
        Each sensor calculates its updated belief function on true  $CO_2$ 
         $P_{t+1}^n(z_{t+1} | y_{1:t}^n, c_{1:t+1})$  using (9) and send it back to the cloud
        ;
        The cloud fusion center computes the approximated belief
        state  $\tilde{\theta}_{t+1}$  according to the fusion rule proposed in Section
        V-C;
        Store the transition  $(\tilde{\theta}_t, b_t, l_t, \tilde{\theta}_{t+1})$  into experience replay
        buffer  $\mathcal{S}$ , if  $\mathcal{S}$  is full, remove the oldest entry in  $\mathcal{S}$ ;
        If the number of the elements in replay buffer is larger than
         $N_D$ , randomly select  $N_D$  samples for the mini-batch calculate
        loss using (33);
        Update the parameter  $w_t$  using (34);
        Update the target  $Q$ -network parameter  $w_t^-$  every  $F$  time steps
        using (34);
    end
    Save both  $w$  and target model parameter  $w^-$ .
end
Output: Reference sensor selection policy according to (35).

```

DQN agent randomly decides on a minibatch D_i of the replay buffer. The objective of DQN is to minimize the following mean square error:

$$L(w_i) = \mathbb{E}_{s,a,r,s' \sim D_i} \left[\left(l(s, a) + \gamma \min_b Q(s', b, w_i^-) - Q(s, a, w_i) \right)^2 \right] \quad (33)$$

where w^- represents the parameters of the target Q -network. It is updated every certain number of training steps, while $\mathbb{E}_{s,a,r,s' \sim D_i}$ indicates the mean square error is calculated by using the samples of transition tuples from the minibatch D_i . The loss is further sent backward through the deep neural network to calculate the gradient with respect to the parameter w_i to update the weight w_i according to

$$w_{i+1} = w_i + \alpha \nabla_{w_i} L(w_i) \quad (34)$$

where $\alpha \in (0, 1]$ is the learning rate. After learning the optimal Q -function, the optimal policy is given by

$$\pi^*(s) = \arg \min_{a \in \{1, 2, \dots, \tilde{N}\}} \hat{Q}^*(s, a, w) \quad (35)$$

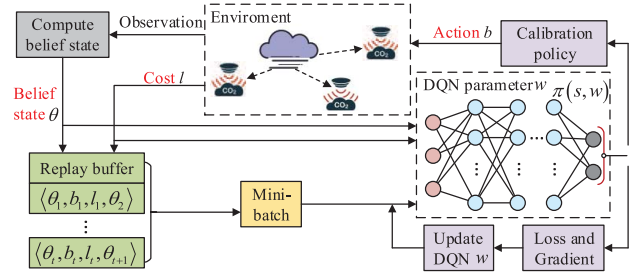


Fig. 3. System model of DQN-based networked calibration for an IoT-based CO_2 concentration monitoring system. A storage device is equipped at the cloud that acts as the replay buffer for the DQN algorithm, while several computing nodes are also equipped at the cloud that enables computation of belief state, training of DQN, and the calibration policy design performed by the DQN agent.

where $\hat{Q}^*(s, a, w)$ is the optimal approximated Q -function. Combining the DQN framework, the process of our sensor self-calibration is presented in Algorithm 1, and the design of our IoT-based CO_2 concentration monitoring system is illustrated in Fig. 3.

VI. NUMERICAL RESULTS

In this section, we present numerical results of our proposed calibration frameworks. In the simulation, the software environment is Python 3.6 with Keras 2.4.3, which has been widely used to deploy machine learning algorithms and experiment environments. The experiments are carried out based on the data collected from the sensors integrated at the air quality monitoring station in Dübendorf in Switzerland.⁴ The data set contains record from April 2017 to December 2019, with the data collected from 18 Senseair LP8 sensors as well as a co-located Picarro G1301 reference sensor. In more details, the following quantities are utilized in our experiments: 1) true values of CO_2 concentration level and the temperature at each minute during the whole time span are provided by the reference sensor and 2) measurements of CO_2 concentration level and corresponding sensor current of the detector provided by LP8 sensors and the data before 2017.07.01 are single measurements, data after 2017.07.01 are 10-min average values.

A. Numerical Results for HMM-Based Stochastic Modeling Framework for Single Sensor

In this experiment, we first train the HMM for the single sensor, i.e., learning the HMM parameters, based on the supervised learning approach as described in Section IV-B. Based on the derived HMM parameters, each single sensor can then update the belief function of its true calibration parameter according to Lemma 1. The calibration becomes the problem of inferring the most likely true calibration parameter based on the updated belief function, i.e., find the maximum a posteriori (MAP) estimation of the current true calibration parameter based on the updated belief function. In our experiment, we use the data from five sensors and learn the HMM parameters

⁴The sensor network is deployed by EMPA.

for each sensor. The performance of the calibration framework is also tested for each sensor. Since the data before January 2018 contain several long periods without any recordings, we use data collected from 2018.01.18 to 2018.10.18 to train the HMM for each sensor, and use the data from 2018.10.20 to 2019.03.21 to test the performance. The sampling interval is set to 1 h to ensure that the time series are strictly synchronized. Given the true CO_2 concentration level, the sensor current of the detector, the temperature, and other fixed sensor parameters, we can first derive the true values of calibration parameters by solving (2). Within the training data set, the CO_2 measurement, the true calibration parameter, and the sensor current of the detector are quantized to certain integer values within different ranges. Furthermore, we quantize the change of temperature to three different levels.

- 1) *Temperature Decreases*: The temperature change lies in the region $(-\infty, -0.5)$.
- 2) *Temperature Remains the Same*: The temperature change lies in the region $[-0.5, 0.5]$.
- 3) *Temperature Increases*: The temperature change lies in the region $(0.5, +\infty)$.

The performance of our HMM-based calibration framework is presented in Fig. 4, where we compare the predicted CO_2 concentration level [derived by solving (2) given predicted true calibration parameter and other parameters] to the true CO_2 concentration level during three different periods from 2018.10.20 to 2019.03.21. Based on the figures, we can easily observe that the HMM for each sensor performs well in the beginning. The prediction is made only three days after the model has been trained; thus, the drift process behaves quite similar with what the sensors have experienced during the training phase. However, the temperature dependency of the drift behavior cannot be perfectly captured by the HMM due to the remaining model errors. Thus, the aggregation of the imperfectness will lead to the degradation of the HMM performance over time, because more and more drift behaviors that have not been experienced during training phase will appear. As we can see from the figures, the performance of the HMM for sensors 1 and 2 starts to degrade three months after the models have been trained, and the performance of the HMM for sensor 4 remains very good after six months.

B. Numerical Results for Networked Calibration

In this section, we consider a network with the five sensors mentioned above (with computation ability). It is also equipped with a computing node that serves as the fusion center (for belief function fusion calibration) or the DQN agent (for DQN-based calibration), and the sensors can communicate with the computing node.

1) *Results for Belief Function Fusion-Based Calibration*: We first provide the numerical results for our Wasserstein distance-based weighted average belief function fusion approach. We fuse the belief functions of all five sensors in both period 2019.01.03–2019.01.10 and 2019.03.21–2019.03.27. The predicted CO_2 level is given by the MAP estimation performed based on the fusion result. As it can be observed from Fig. 4, the belief function of the true CO_2 level

TABLE I
SETTING OF PARAMETERS FOR DQN ENVIRONMENT

Parameters	Value
Number of sensors	5
Number of DQN agent	1
Temperature ($^{\circ}C$)	-5:0.1:24
True CO_2 level (ppm)	250:1:750

provided by sensors 3, 4, and 5 is consistent and reliable during the period 2019.01.03–2019.01.10. In this case, our proposed weighted average-based belief function fusion approach will assign a high weight to the belief functions from sensors 3, 4, and 5, thus leading to an accurate and correct fused belief function. On the other hand, during the period 2019.03.21–2019.03.27, only the belief function of sensor 4 provides the correct evidence. In this case, higher weight will be assigned to the other belief functions, which provide incorrect evidence, and the performance of our proposed belief function fusion approach degrades rapidly. The above results are illustrated in Fig. 5.

2) *Numerical Results for Deep Q-Network-Based Calibration for Sensor Network*: To test the performance of our proposed DRL algorithm, we have to build a simulation environment that can interact with the DQN agent. The code for building this simulation environment is provided at [41]. On receiving an action from the DQN agent, i.e., decision on the reference sensor, the environment can simulate the evolution of the belief state according to the process as described in Section V-D1 and generates a reward that is sent back to the DQN agent. Since our algorithm is built upon a predefined probabilistic space, we restrict the belief functions of the true calibration parameter and true CO_2 levels from different sensors to be defined on the same support set. In more details, the true calibration parameter for each sensor can only take certain integer values within the same range. The settings of the true CO_2 level and the environmental temperature are presented in Table I, while the CO_2 measurements and the sensor current of the detector for different sensors are again quantized to certain integer values within certain ranges.

The selection of the network parameters decides the learning convergence speed and efficiency. In this article, we specifically illustrate the impact of different values of learning rate on the performance of our DQN algorithm. Fig. 6 shows the average calibration cost versus training episodes under different learning rates, i.e., $\alpha = \{0.1, 0.01, 0.001\}$. It can be observed that it requires longer convergence time when the learning rate decreases. However, fast convergence would probably make the agent fall into a local optimum and miss the better strategy. Based on the above results, we set our learning rate that equals 0.001.

For the DQN agent, we use a 5-layer perceptron where we have three connected hidden layers containing 500, 300, and 200 neurons, respectively. The learning rate is set to $\alpha = 0.001$, the discount factor is set to $\gamma = 0.99$, and the exploration rate ϵ is initially set as 0.9 and decays to 0.001 at rate 0.995. The DQN model is trained for 2000 epochs under

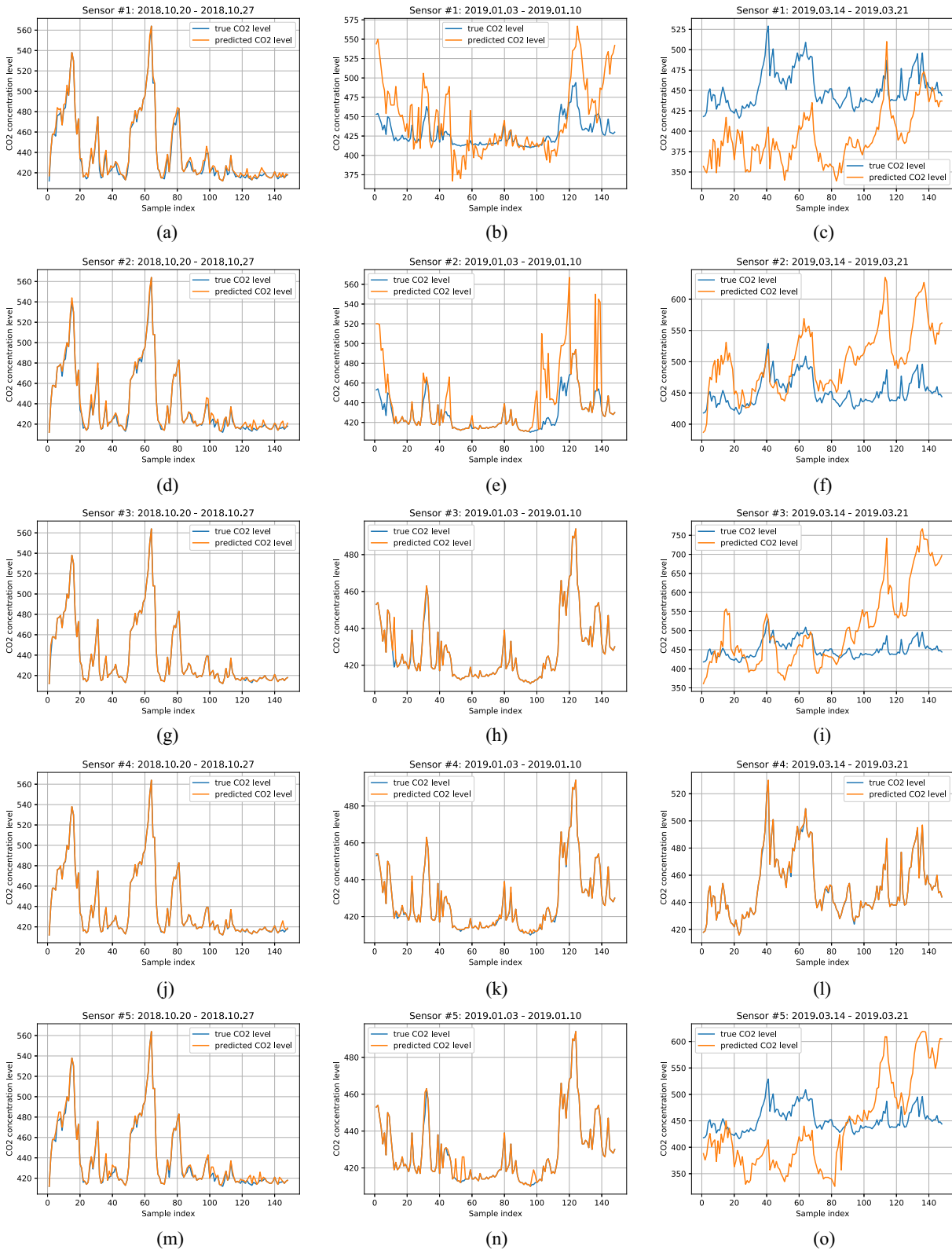


Fig. 4. Predicted true CO_2 level in different periods from 2018.10.20 to 2019.03.21 using HMM-based framework. (a)–(c) Prediction results for sensor 1. (d)–(f) Prediction results for sensor 2. (g)–(i) Prediction results for sensor 3. (j)–(l) Prediction results for sensor 4. (m)–(o) Prediction results for sensor 5.

a experience replay with memory size 50 000. The minibatch size is set as 64 and the target Q network is updated every 500 time steps.

With the trained DQN agent, we test our DQN-based calibration algorithm during period 2019.01.03–2019.03.21.

The performance is illustrated in Fig. 7. As it can be seen from Fig. 7(a), our proposed DQN-based calibration algorithm performs worse than our modified belief function fusion approach in the beginning of the testing period. This is the case since the DQN-based algorithm provides a long-term

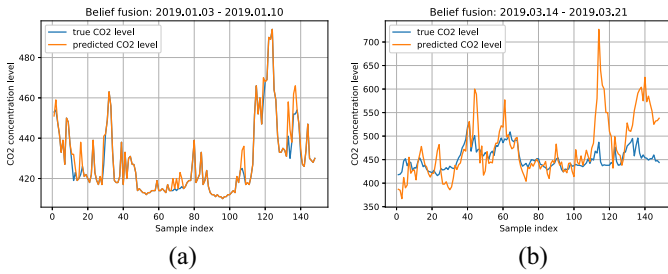


Fig. 5. Predicted true CO_2 level using belief function fusion approach. (a) Prediction result from 2019.01.03 to 2019.01.10. (b) Prediction result from 2019.03.14 to 2019.03.21.

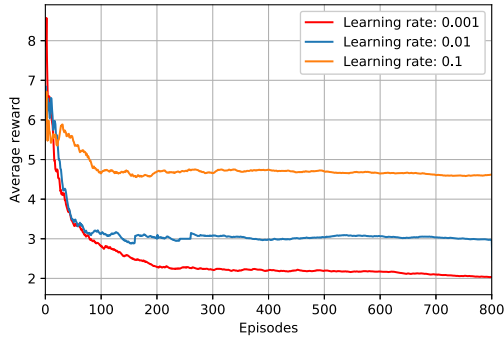


Fig. 6. Average cost performance versus episodes under different learning rates.

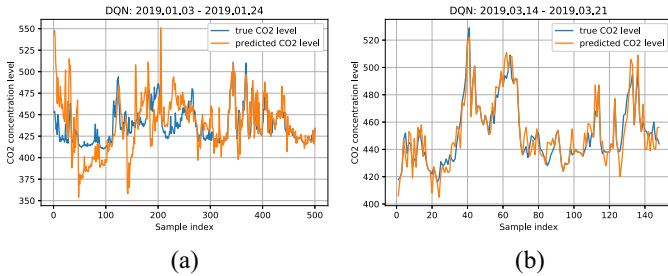


Fig. 7. Predicted true CO_2 level using DQN-based calibration. (a) Prediction result from 2019.01.03 to 2019.01.10. (b) Prediction result from 2019.03.14 to 2019.03.21.

calibration strategy instead of calibrating the sensors according to the best instantaneous choice. When most of the sensors have a well-performing single sensor HMM model, the modified belief function fusion approach can thus outperform the DQN-based calibration algorithm. As it can be also observed, the performance illustrated in Fig. 7(a) improves over time and finally, ends up with a high-accuracy performance as illustrated in Fig. 7(b). In comparison, the performance of modified belief function fusion degrades with the time as shown in Fig. 5(b). This means that even with some drawbacks under some specific cases, the DQN-based calibration algorithm has a stable and good performance over the long term.

VII. CONCLUSION

The development of the IoT-based gas concentration monitoring systems is helping to better monitor the air quality. However, to keep the cost low for sensor deployment, one often uses sensors with components that suffer from unknown dependencies on environmental factors. These

unknown dependencies between the environmental factors and the sensor components lead to lower accuracy. Thus, with additional calibration, better performance of the whole gas monitoring system can be achieved.

In this article, we developed several self-calibration algorithms for IoT-based gas concentration monitoring systems. The measurement errors of the sensors in the system are mainly caused by the remaining model errors that can be fully described by the drift of the calibration parameter. This leads to our first formulation of a statistical inference problem on the true calibration parameter under the HMM framework. When considering the networked calibration for the whole IoT-based system, the proposed weighted average belief function fusion approach can be used to deal with the case when belief functions provided by different sensors highly conflict with each other. The corresponding fusion result can be used as an approximation of the joint statistics when it is difficult to form a joint statistics among different sensors. As a consequence, the proposed POMDP-based multisensor sequential self-calibration approach utilizes the fusion result as an approximation of the joint statistics, and achieves more stable long-term calibration performance. Based on the numerical experiments, the weighted average belief function fusion approach is shown to have higher calibration accuracy than the POMDP-based approach when the majority of the sensors beliefs provide reliable and correct evidence. On the other hand, the POMDP-based approach can achieve mild but stable calibration accuracy over a relatively long period compared to the weighted average belief function fusion approach.

Since there will be remaining model errors that cannot be fully captured by the proposed, every calibrated model will become out-of-date at some point of time. Future work is therefore to identify the optimal frequency a model should be updated and how this can be done efficiently, e.g., using a time-adaptive model.

ACKNOWLEDGMENT

The authors like to acknowledge Henrik Rödjegård, Hans Martin, and Floria Ottonello Briano from Senseair for the helpful technical discussions and for providing us the sensor data used in this work.

REFERENCES

- [1] P. Ferrer-Cid, J. M. Barcelo-Ordinas, J. Garcia-Vidal, A. Ripoll, and M. Viana, "Multisensor data fusion calibration in IoT air pollution platforms," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3124–3132, Apr. 2020.
- [2] B. Maag, Z. Zhou, and L. Thiele, "A survey on sensor calibration in air pollution monitoring deployments," *IEEE Internet Things J.*, vol. 5, no. 6, pp. 4857–4870, Dec. 2018.
- [3] J. B. A. Gomes, J. J. P. C. Rodrigues, R. A. L. Rabêlo, N. Kumar, and S. Kozlov, "IoT-enabled gas sensors: Technologies, applications, and opportunities," *J. Sensor Actuator Netw.*, vol. 8, no. 4, p. 57, 2019. [Online]. Available: <https://www.mdpi.com/2224-2708/8/4/57>
- [4] J. B. A. Gomes, J. J. P. C. Rodrigues, J. Al-Muhtadi, N. Arunkumar, R. A. L. Rabêlo, and V. Furtado, "An IoT-based smart solution for preventing domestic CO and LPG gas accidents," in *Proc. IEEE 10th Latin-Amer. Conf. Commun. (LATINCOM)*, 2018, pp. 1–6.
- [5] "Why NDIR?" [Online]. Available: <https://senseair.com/knowledge/sensor-technology/technology/why-ndir/> (accessed Oct. 19, 2019).
- [6] J. Park, H. Cho, and S. Yi, "NDIR CO_2 gas sensor with improved temperature compensation," *Procedia Eng.*, vol. 5, pp. 303–306, Sep. 2010. [Online]. Available: <https://doi.org/10.1016/j.proeng.2010.09.108>

- [7] M. Müller *et al.*, "Integration and calibration of non-dispersive infrared (NDIR) CO₂ low-cost sensors and their operation in a sensor network covering Switzerland," *Atmos. Meas. Tech.*, vol. 13, no. 7, pp. 3815–3834, 2020. [Online]. Available: <https://amt.copernicus.org/articles/13/3815/2020/>
- [8] C. R. Martin *et al.*, "Evaluation and environmental correction of ambient CO₂ measurements from a low-cost ndir sensor," *Atmos. Meas. Tech.*, vol. 10, no. 7, pp. 2383–2395, 2017.
- [9] A. Xu, "On stochastic modeling of sensors that drift," KTH Dissertation, School Elect. Eng. Comput. Sci., Eng. Technol., Stockholm Sweden, 2019.
- [10] "Calibration Procedure," SenseAir, Delsbo, Sweden, Rep. TN-011, 2000.
- [11] D. Solomatine, L. M. See, and R. J. Abraham, "Data-driven modelling: Concepts, approaches and experiences," in *Practical Hydroinformatics*. Berlin, Germany: Springer, 2009, pp. 17–30.
- [12] N. Roy and S. Thrun, "Online self-calibration for mobile robots," in *Proc. IEEE Int. Conf. Robot. Autom.*, vol. 3, May 1999, pp. 2292–2297.
- [13] T. Wissel, B. Wagner, P. Stüber, A. Schweikard, and F. Ernst, "Data-driven learning for calibrating galvanometric laser scanners," *IEEE Sensors J.*, vol. 15, no. 10, pp. 5709–5717, Oct. 2015.
- [14] L. Chen *et al.*, "Data-driven calibration of soil moisture sensor considering impacts of temperature: A case study on FDR sensors," *Sensors*, vol. 19, no. 20, p. 4381, 2019.
- [15] D. Wang, J. Liu, and R. Srinivasan, "Data-driven soft sensor approach for quality prediction in a refining process," *IEEE Trans. Ind. Informat.*, vol. 6, no. 1, pp. 11–17, Feb. 2010.
- [16] K. Shafique, A. Hakeem, O. Javed, and N. Haering, "Self calibrating visual sensor networks," in *Proc. IEEE Workshop Appl. Comput. Vis.*, 2008, pp. 1–6.
- [17] L. O. Wijeratne, D. R. Kiv, A. R. Aker, S. Talebi, and D. J. Lary, "Using machine learning for the calibration of airborne particulate sensors," *Sensors*, vol. 20, no. 1, p. 99, 2020.
- [18] W. Xie and P. Bai, "A pressure sensor calibration model based on support vector machine," in *Proc. 24th Chin. Control Decis. Conf. (CCDC)*, 2012, pp. 3239–3242.
- [19] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [20] B. Chen, W.-A. Zhang, and L. Yu, "Distributed finite-horizon fusion Kalman filtering for bandwidth and energy constrained wireless sensor networks," *IEEE Trans. Signal Process.*, vol. 62, no. 4, pp. 797–812, Feb. 2014.
- [21] M. Ghanbari and M. J. Yazdanpanah, "Delay compensation of tilt sensors based on MEMS accelerometer using data fusion technique," *IEEE Sensors J.*, vol. 15, no. 3, pp. 1959–1966, Mar. 2015.
- [22] R. Jassemi-Zargani and D. Neculescu, "Extended Kalman filter-based sensor fusion for operational space control of a robot arm," *IEEE Trans. Instrum. Meas.*, vol. 51, no. 6, pp. 1279–1282, Dec. 2002.
- [23] T. Denoeux and M. Masson, "Belief functions: Theory and applications," in *Proc. 2nd Int. Conf. Belief Funct.*, 2012, pp. 9–11.
- [24] D. Alshamaa, F. Mourad-Chehade, and P. Honeine, "Tracking of mobile sensors using belief functions in indoor wireless networks," *IEEE Sensors J.*, vol. 18, no. 1, pp. 310–319, Jan. 2018.
- [25] D. Alshamaa, F. Mourad-Chehade, and P. Honeine, "Decentralized Kernel-based localization in wireless sensor networks using belief functions," *IEEE Sensors J.*, vol. 19, no. 11, pp. 4149–4159, Jun. 2019.
- [26] N. Milisavljevic and I. Bloch, "Sensor fusion in anti-personnel mine detection using a two-level belief function model," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 33, no. 2, pp. 269–283, May 2003.
- [27] E. Lefevre, O. Colot, and P. Vannooenbergh, "Belief function combination and conflict management," *Inf. Fusion*, vol. 3, no. 2, pp. 149–162, 2002.
- [28] C. K. Murphy, "Combining belief functions when evidence conflicts," *Decis. Support Syst.*, vol. 29, no. 1, pp. 1–9, 2000.
- [29] D. Yong, S. W. Kang, Z. Z. Fu, and L. Qi, "Combining belief functions based on distance of evidence," *Decis. Support Syst.*, vol. 38, no. 3, pp. 489–493, 2004.
- [30] Z. Zhang, T. Liu, D. Chen, and W. Zhang, "Novel algorithm for identifying and fusing conflicting data in wireless sensor networks," *Sensors*, vol. 14, no. 6, pp. 9562–9581, 2014.
- [31] Y. Yang and D. Han, "A new distance-based total uncertainty measure in the theory of belief functions," *Knowl. Based Syst.*, vol. 94, pp. 114–123, Feb. 2016.
- [32] A.-L. Jousselme, D. Grenier, and É. Bossé, "A new distance between two bodies of evidence," *Inf. Fusion*, vol. 2, no. 2, pp. 91–101, 2001.
- [33] C. Villani, *The Wasserstein Distances*. Berlin, Germany: Springer, 2009, pp. 93–111. [Online]. Available: https://doi.org/10.1007/978-3-540-71050-9_6
- [34] V. Krishnamurthy, *Partially Observed Markov Decision Processes: From Filtering to Controlled Sensing*. Cambridge, U.K.: Cambridge Univ. Press, 2016.
- [35] M. Cao, J. Chen, and J. Wang, "A novel vehicle tracking method for cross-area sensor fusion with reinforcement learning based GMM," in *Proc. Amer. Control Conf. (ACC)*, 2020, pp. 442–447.
- [36] R. Sutton and A. Barto, *Introduction to Reinforcement Learning*, vol. 2. Cambridge, MA, USA: MIT Press, 1998.
- [37] K. C. Tan, K. K. Tan, T. Lee, S. Zhao, and Y. Chen, "Autonomous robot navigation based on fuzzy sensor fusion and reinforcement learning," in *Proc. IEEE Int. Symp. Intell. Control*, 2002, pp. 182–187.
- [38] Y. You and T. J. Oechtering, "Hidden Markov model based data-driven calibration of non-dispersive infrared gas sensor," in *Proc. 28th Eur. Signal Process. Conf. (EUSIPCO)*, 2021, pp. 1717–1721.
- [39] F. Zhao, J. Shin, and J. Reich, "Information-driven dynamic sensor collaboration," *IEEE Signal Process. Mag.*, vol. 19, no. 2, pp. 61–72, Mar. 2002.
- [40] V. Mnih *et al.*, "Playing Atari with deep reinforcement learning," in *Proc. NIPS Deep Learn. Workshop*, 2013, *arXiv:1312.5602*.
- [41] Accessed: Jan. 27, 2022. [Online]. Available: <https://github.com/tmacyou/DRL-Environment-Sensor-Calibration.git>



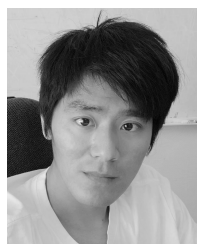
Yang You (Student Member, IEEE) received the B.Eng. degree in information engineering from Beijing Institute of Technology, Beijing, China, in 2014, and the M.Sc. degree in electrical engineering from the KTH Royal Institute of Technology, Stockholm, Sweden, in 2017, where he is currently pursuing the Ph.D. degree with the Division of Information Science and Engineering.

His research interests include statistical signal processing, stochastic optimal control, and information security and privacy.



Kai You is currently pursuing the B.Eng. degree in computer science and technology with Shanghai Jiao Tong University, Shanghai, China.

His research interests include statistical learning, mobile computing, and data management.



Hao Chen (Student Member, IEEE) received the B.Sc. degree in communication engineering and the M.Sc. degree in electronic and communication engineering from Soochow University, Suzhou, China, in 2014 and 2017, respectively. He is currently pursuing the Ph.D. degree with the School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology, Stockholm, Sweden.

His current research interests include distributed machine learning, distributed optimization, and edge computing.



Tobias J. Oechtering (Senior Member, IEEE) received the Dipl.-Ing. degree in electrical engineering and information technology from RWTH Aachen University, Aachen, Germany, in 2002, and the Dr.-Ing. degree in electrical engineering from Technische Universität Berlin, Berlin, Germany, in 2007.

In 2008, he joined the KTH Royal Institute of Technology, Stockholm, Sweden and has been a Professor since 2018. His research interests include statistical signal processing, communication and information theory, physical-layer privacy and security, as well as communication for networked control.

Prof. Oechtering received the "Förderpreis 2009" from the Vodafone Foundation in 2009. He is currently a Senior Editor of IEEE TRANSACTIONS ON INFORMATION FORENSIC AND SECURITY since May 2020 and has been served previously as an Associate Editor for the IEEE TRANSACTIONS ON INFORMATION FORENSIC AND SECURITY since June 2016, and IEEE COMMUNICATIONS LETTERS from 2012 to 2015. He has served on numerous technical program committees for IEEE sponsored conferences, and he was a General Chair for IEEE ITW 2019.